

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #38 — Dropshipping Inventory & Order Orchestrator

1. Requirements Table

Target Stakeholders / Actors: E-Store Owner, Supplier Partner, Storefront (Shopify / WooCommerce webhook source).

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse inbound Shopify/WooCommerce order webhooks, verify supplier API inventory, and auto-dispatch supplier purchase orders.	High	Pass: Supplier PO triggered with customer delivery address. Fail: Order placed for out-of-stock supplier SKU.	Core order-to-fulfillment path; prevents overselling against real-time supplier stock.
FR-002	Functional	The system shall split a single customer order containing SKUs from multiple Supplier Partners into discrete per-supplier purchase orders.	High	Pass: An order with items from 2 distinct suppliers generates exactly 2 POs, each containing only that supplier's SKUs. Fail: A single PO contains SKUs from more than one supplier.	Suppliers fulfill and ship independently; a consolidated PO cannot be processed by a single vendor.
FR-003	Functional	The system shall ingest tracking numbers submitted by Supplier Partners (via API push or manual upload) and associate each tracking number with its correct order line item.	Medium	Pass: Tracking number is attached to the correct line item within 5 minutes of supplier submission. Fail: Tracking number is unassigned or linked to the wrong order.	Enables order status visibility without manual reconciliation.
FR-004	Functional	The system shall maintain a SKU mapping table translating storefront product SKUs into the corresponding Supplier Partner catalog SKUs.	High	Pass: Given a storefront SKU with an active mapping, the system resolves to exactly one supplier SKU. Fail: Order is processed against an unmapped or ambiguous SKU.	Storefront and supplier catalogs use independent identifier schemes; mapping is a prerequisite for correct PO generation.
FR-005	Functional	The system shall update order status (Pending → Dispatched → Shipped → Delivered) and trigger a customer-facing notification on each transition.	Medium	Pass: Status change is logged with a timestamp and notification is sent within 2 minutes. Fail: Dashboard status and notification are out of sync.	Maintains fulfillment transparency for the E-Store Owner and end customer.
NFR-001	Nonfunctional (Performance & Security)	Inbound webhook processing must acknowledge receipt with HTTP 200 within 200 ms to prevent provider webhook retry storms.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Webhook providers retry aggressively on slow/failed acknowledgment; unmet SLA causes duplicate deliveries.
NFR-002	Nonfunctional (Reliability / Idempotency)	The system shall process each inbound webhook exactly once, using idempotency keys derived from the order/webhook ID, even under duplicate delivery.	High	Pass: Sending an identical webhook payload twice results in exactly one dispatched PO and one inventory deduction. Fail: Duplicate delivery produces a duplicate PO or double inventory deduction.	Shopify/WooCommerce retry on timeout (NFR-001); without idempotency this causes duplicate supplier orders and billing errors.

2. Use-Case Flow Specification: “Dispatch Purchase Order”

Actors

Primary: E-Store Owner. Secondary: Supplier Partner. Trigger source: Storefront (Shopify / WooCommerce).

Preconditions

- Storefront webhook has been received, signature-validated, and acknowledged with HTTP 200 (NFR-001).
- Order line items resolve to at least one active Supplier Partner via the SKU mapping table (FR-004).
- Supplier Partner API credentials are valid and the endpoint is reachable.

Postconditions (Success)

- One or more Supplier Purchase Orders are created and transmitted to the corresponding Supplier Partner API(s).
- Order status is updated to “Dispatched,” with a supplier order ID recorded per PO.
- Inventory reservation is logged against each dispatched SKU.

Main Success Scenario

1. Storefront delivers a new order payload to the webhook ingestion endpoint.
2. System validates the webhook signature and returns HTTP 200 within the 200 ms SLA (NFR-001).
3. System parses the payload, extracting line items, quantities, and delivery address.
4. System resolves each storefront SKU to its Supplier Partner SKU via the mapping table (FR-004).
5. System groups line items by Supplier Partner — «include» Split Multi-Vendor Order (FR-002).
6. For each supplier group, system queries the Supplier Partner’s inventory API — «include» Verify Supplier Inventory.
7. Stock is confirmed available for all SKUs in the group.
8. System generates a PO payload containing confirmed SKUs and the customer delivery address.
9. System dispatches the PO to the Supplier Partner API.
10. Supplier Partner API returns a PO acknowledgment with a supplier order ID.
11. System records the supplier order ID and updates order status to “Dispatched” (FR-005).
12. Use case ends successfully.

Alternate Flow — 7a. Supplier API Connection Failure

- 7a1. The inventory API call times out or returns a connection error.
- 7a2. System retries with exponential backoff, up to 3 attempts.
- 7a3. If all retries fail, the affected supplier group is flagged “Supplier Unreachable”; PO dispatch is withheld for that group only — unaffected supplier groups from the same order continue independently.
- 7a4. Failure is logged with timestamp and error code; E-Store Owner is notified via dashboard alert.
- 7a5. Once connectivity is restored, the E-Store Owner (or a scheduled retry job) resumes the flow at step 6.